

Inspecionando fluxo HTTP OAuth2/PKCE

quinta-feira, 10 de abril de 2025 14:40

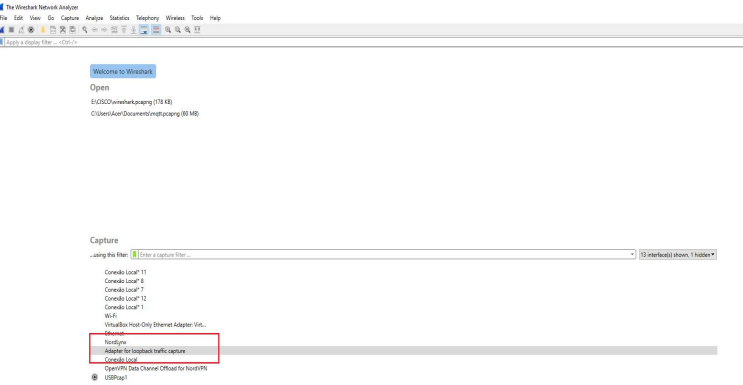
SOBRE ESTE DOCUMENTO

Este documento é uma análise técnica e pedagógica sobre o uso do Wireshark para monitoramento e entendimento do Fluxo OAuth2 com PKCE, utilizando Keycloak como Authorization Server e o Postman como cliente OAuth2. O ambiente é executado localmente (localhost), e o tráfego de autenticação é monitorado diretamente pela interface de loopback com Wireshark.

Durante o estudo, são analisadas requisições HTTP antes, durante e após o processo de autenticação, com foco nos frames capturados, estrutura dos tokens, parâmetros PKCE (code_verifier, code_challenge) e o comportamento do Keycloak como servidor de autorização.

INTRODUÇÃO

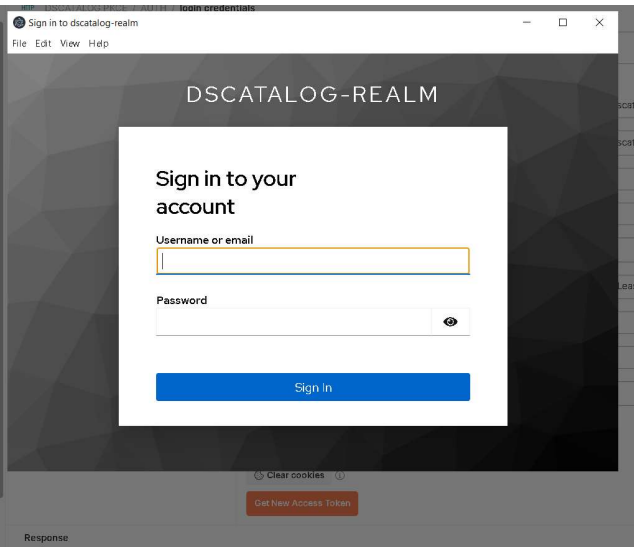
Como meu sistema está em Localhost tenho que monitorar o tráfego de loopback (Adapter for Loopback traffic capture) esta interface pode estar escondida, desabilite para mostrar todas as interfaces. A partir da versão 3.4 do Wireshark, é possível capturar o tráfego de loopback diretamente.



- ☑ O tráfego HTTP está passando pela interface loopback via ::1 (IPv6)
- ☑ Use o filtro `(http.request || http.response) && (ip.addr == 127.0.0.1 || ipv6.addr == ::1)`
- ☑ O fluxo de autenticação Authorization Code + PKCE gera requisições visíveis (mesmo com HTTPS criptografado, dá pra ver estrutura)

TRAFEGO HTTP ANTES DE SUBMETER AS CREDENCIAIS VIA POSTMAN

- 1. Clico no botão "Get New Access Token"



NO WIRESHARK

No.	Time	Source	Destination	Protocol	Length	Info
44	30.4112255	::1	::1	HTTP	910	GET /realms/dscatalog-realm/protocol/openid-connect/auth?response_type=code&client_id=dscatalog-client&scope=openid&redirect_uri=http%3A%2F%2Flocalhost%3A8080%2F... HTTP/1.1
51	39.769767	::1	::1	HTTP	3714	HTTP/1.1 200 OK (text/html)
56	39.782928	::1	::1	HTTP	609	GET /resources/a9dep/common/keycloak/vendor/patternfly-v5/patternfly.min.css HTTP/1.1
58	39.783415	::1	::1	HTTP	612	GET /resources/a9dep/common/keycloak/vendor/patternfly-v5/patternfly-addons.css HTTP/1.1
63	39.785533	::1	::1	HTTP	586	GET /resources/a9dep/login/keycloak-v2/css/styles.css HTTP/1.1
68	39.786574	::1	::1	HTTP	610	GET /resources/a9dep/login/keycloak-v2/js/passwordVisibility.js HTTP/1.1
73	39.793861	::1	::1	HTTP	682	GET /resources/a9dep/login/keycloak-v2/js/authChecker.js HTTP/1.1
75	39.815384	::1	::1	HTTP	2420	HTTP/1.1 200 OK (text/css)
76	39.815384	::1	::1	HTTP	2690	HTTP/1.1 200 OK (text/javascript)
77	39.815415	::1	::1	HTTP	1018	HTTP/1.1 200 OK (text/javascript)
107	39.832914	::1	::1	HTTP	18464	HTTP/1.1 200 OK (text/css)
196	39.880920	::1	::1	HTTP	38875	HTTP/1.1 200 OK (text/css)
197	39.987318	::1	::1	HTTP	637	GET /resources/a9dep/login/keycloak-v2/img/keycloak-bg.png HTTP/1.1
205	39.916266	::1	::1	HTTP	20530	HTTP/1.1 200 OK (PNG)
207	39.943643	::1	::1	HTTP	651	GET /resources/a9dep/common/keycloak/vendor/patternfly-v5/assets/fonts/RedHatText/RedHatText-Regular.woff2 HTTP/1.1
209	39.943919	::1	::1	HTTP	656	GET /resources/a9dep/common/keycloak/vendor/patternfly-v5/assets/fonts/RedHatDisplay/RedHatDisplay-Medium.woff2 HTTP/1.1
211	39.949822	::1	::1	HTTP	650	GET /resources/a9dep/common/keycloak/vendor/patternfly-v5/assets/fonts/RedHatText/RedHatText-Medium.woff2 HTTP/1.1
213	39.951197	::1	::1	HTTP	643	GET /resources/a9dep/common/keycloak/vendor/patternfly-v5/assets/fonts/RedHatText/RedHatText-Medium.woff2 HTTP/1.1
221	39.958350	::1	::1	HTTP	14477	HTTP/1.1 200 OK
225	39.962347	::1	::1	HTTP	22128	HTTP/1.1 200 OK
229	39.966217	::1	::1	HTTP	23164	HTTP/1.1 200 OK
235	39.966790	::1	::1	HTTP	54652	HTTP/1.1 200 OK
265	92.937329	127.0.0.1	127.0.0.1	HTTP	181	GET /asupdate HTTP/1.0
279	92.938293	127.0.0.1	127.0.0.1	HTTP	82	HTTP/1.0 200

Participantes:

- Postman: Aplicativo cliente OAuth2 (navegador embutido).
- Keycloak: Authorization Server (protocolo OpenID Connect).

- Loopback (::1 / 127.0.0.1): Toda comunicação está acontecendo localmente via IPV6 (no mesmo host).

Etapas capturadas (análise do tráfego):

1. GET /auth?...

GET /realms/dscatalog-realm/protocol/openid-connect/auth?response_type=code...

Essa é a requisição inicial do fluxo **Authorization Code** com PKCE.

O Postman está pedindo autorização para um usuário, e fornece:

- response_type=code
- client_id=dscatalog-client
- scope=openid
- redirect_uri=http://localhost:8080/...
- code_challenge=...
- code_challenge_method=S256 (veja que o code_challenge é o

```
> Frame 44: 910 bytes on wire (7280 bits), 910 bytes captured (7280 bits) on interface \Device\NPF_{...} id 0
> Null/Loopback
> Internet Protocol Version 6, Src: ::1, Dst: ::1
> Transmission Control Protocol, Src Port: 19372, Dst Port: 8081, Seq: 1, Ack: 1, Len: 846
> Hypertext Transfer Protocol
  > [truncated]GET /realms/dscatalog-realm/protocol/openid-connect/auth?response_type=code&client_id=dscatalog-client&scope=openid&redirect_uri=http%3A%2F%2Flocalhost%3A8080%2F*&code_challenge=z7cfffSHPPrD1MdEtqCvaOu0ozmDP42PGjxly0kxjUo
    > [truncated]Expert Info (Chat/Sequence): GET /realms/dscatalog-realm/protocol/openid-connect/auth?response_type=code&client_id=dscatalog-client&scope=openid&redirect_uri=http%3A%2F%2Flocalhost%3A8080%2F*&code_challenge=z7cfffSHPPrD1MdEtqCvaOu0ozmDP42PGjxly0kxjUo
      Request Method: GET
    > Request URI [truncated]: /realms/dscatalog-realm/protocol/openid-connect/auth?response_type=code&client_id=dscatalog-client&scope=openid&redirect_uri=http%3A%2F%2Flocalhost%3A8080%2F*&code_challenge=z7cfffSHPPrD1MdEtqCvaOu0ozmDP42PGjxly0kxjUo
      Request URI Path: /realms/dscatalog-realm/protocol/openid-connect/auth
    > Request URI Query: response_type=code&client_id=dscatalog-client&scope=openid&redirect_uri=http%3A%2F%2Flocalhost%3A8080%2F*&code_challenge=z7cfffSHPPrD1MdEtqCvaOu0ozmDP42PGjxly0kxjUo
      Request URI Query Parameter: response_type=code
      Request URI Query Parameter: client_id=dscatalog-client
      Request URI Query Parameter: scope=openid
      Request URI Query Parameter: redirect_uri=http%3A%2F%2Flocalhost%3A8080%2F*
      Request URI Query Parameter: code_challenge=z7cfffSHPPrD1MdEtqCvaOu0ozmDP42PGjxly0kxjUo
      Request URI Query Parameter: code_challenge_method=S256
    > Request Version: HTTP/1.1
```

Nota: veja que ele manda o hascode do CODE VERIFIER no campo code_challenge. O objetivo com essa requisição é iniciar o fluxo de login + autorização, com PKCE ativo.

2. HTTP 200 OK (#51)

HTTP/1.1 200 OK (text/html) - resposta ao frame #44

Keycloak responde com uma página HTML contendo o formulário de login.

3. Requisições GET para recursos estáticos

Ex:

GET /resources/a9dep/common/keycloak/vendor/patternfly-v5/patternfly.min.css HTTP/1.1

O navegador do Postman está renderizando a interface de login do Keycloak. Por isso ele solicita os seguintes recursos:

- CSS: estilos visuais
- JS: comportamento dinâmico (validação de campos, etc.)
- PNG: imagens de fundo ou ícones
- WOFF2: fontes customizadas

O Keycloak está exportando a tela de login para o Postman. Ou seja, o frontend de autenticação é do Keycloak, e o Postman apenas exibe via navegador embutido.

4. GET /asupdate e HTTP/1.0 200

GET /asupdate HTTP/1.0

Esse é provavelmente um keep-alive ou ping feito pelo próprio navegador embutido do Postman ou pela UI do Keycloak, não faz parte do fluxo principal de autenticação.

Fluxo atual está em:

POSTMAN => [GET /auth?...code_challenge=...] => KEYCLOAK

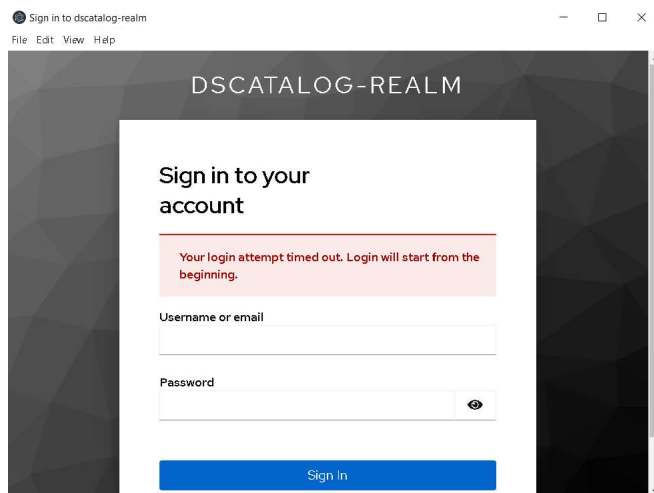
KEYCLOAK => [200 OK com HTML] => POSTMAN

POSTMAN => [GET arquivos estáticos] => KEYCLOAK

O usuário ainda não submeteu login/senha, então:

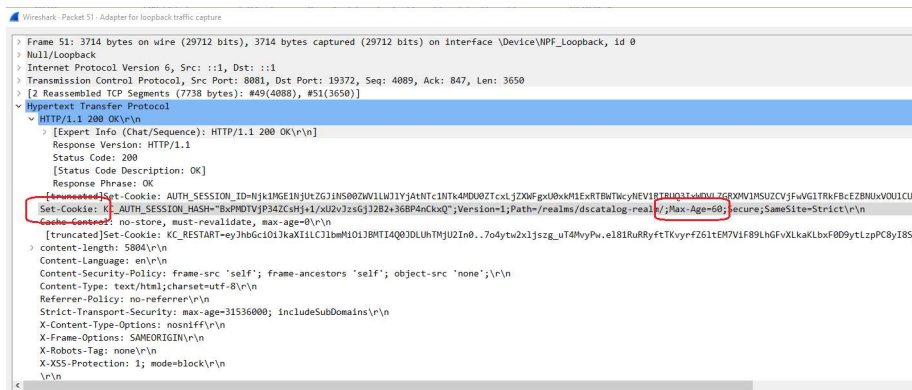
- Nenhum POST /authenticate
- Nenhum authorization_code
- Nenhum POST /token
- Nenhum access_token gerado ainda

Eu demorei demais para entrar com as credenciais de login:



Keycloak define um timeout para a sessão de autenticação (por padrão, 60 segundos para AUTH_SESSION_ID), e se o usuário demorar demais para preencher o formulário de login, o back-end considera a sessão inválida por expiração.

No log você também vê o Set-Cookie com KC_AUTH_SESSION_HASH e Max-Age=60, indicando esse tempo de expiração curto.



A tela de login é reiniciada e as credenciais login/senha são submetidas:

3183	2808.716353	::1	::1	HTTP	23164	HTTP/1.1 200 OK
3199	2808.726715	::1	::1	HTTP	1370	HTTP/1.1 200 OK
3253	2906.764247	::1	::1	HTTP	110	POST /realms/dscatalog-realm/login-actions/authenticate?session_code=dkigKvNlFXvQCKmdRXQZDJJAuF7oh6VXr19sEMlkbT4&execution=09d46113-e83f-436d-8ec4-f01a769e43c0&client_id=dscatalog-client&tab_id=V9pQDPi
3255	2907.393634	::1	::1	HTTP	1568	HTTP/1.1 302 Found
3287	2907.757968	::1	::1	HTTP	697	POST /realms/dscatalog-realm/protocol/openid-connect/token HTTP/1.1 (application/x-www-form-urlencoded)
3289	2907.888891	::1	::1	HTTP/JSON	3810	HTTP/1.1 200 OK , JavaScript Object Notation (application/json)

Etapas após o envio das credenciais (Login)

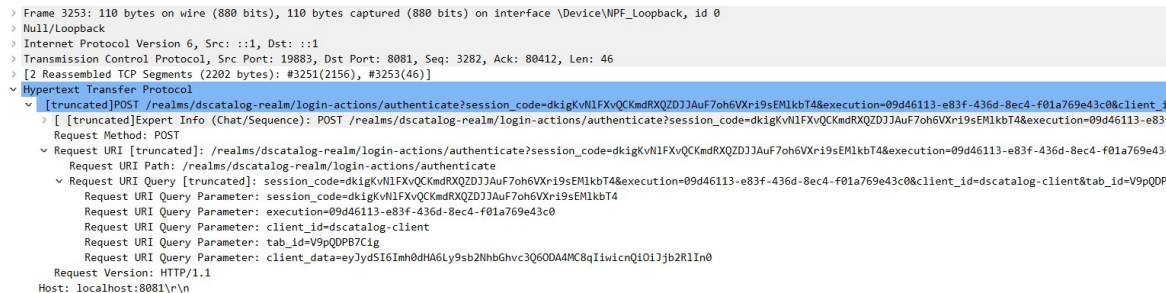
Frame #3253 – POST /login-actions/authenticate

Esse é o momento em que você submeteu o formulário de login com usuário e senha.

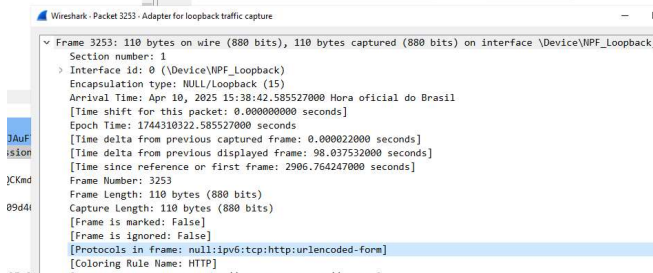
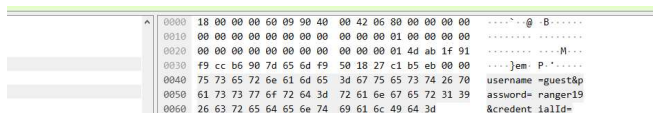
O que é enviado?

- Um POST para a URL de autenticação do Keycloak
- Parâmetros como:
 - session_code
 - execution
 - client_id
 - client_data (com info codificada sobre a origem do fluxo)

Os dados sensíveis como usuário e senha estão no corpo (Content-Type: application/x-www-form-urlencoded), mas neste caso, não aparecem no resumo porque o Wireshark truncou por padrão.



NOTA: Como nosso trafego é HTTP puro e não HTTPS, além disso Content-Type: application/x-www-form-urlencoded, a senha e o login estão como plain text no corpo da requisição POST



Para ambientes reais, obviamente HTTPS é obrigatório, principalmente em fluxos com password, authorization_code, etc.

Frame 3255 – HTTP 302 Found

Keycloak valida as credenciais e, se estiverem corretas, gera um authorization_code e redireciona o cliente (Postman) para o redirect_uri, neste caso:

http://localhost:8080/?session_state=...&code=4409d8fc-119a-454c-999f-6d9c9b44ba6c...

Esse code é válido por pouco tempo (geralmente 1 min) e é trocado imediatamente pelo token.

⚠ IMPORTANTE:

Uma observação neste momento (particularidade do uso do PostMan como Client e não um app real). Em nosso caso o Postman está atuando como cliente OAuth2 completo.

O Postman:

- Escuta internamente a redirect_uri via um navegador embutido
- Intercepta o code da URL recebida (mesmo que não exista de fato um servidor escutando)
- E logo em seguida faz o POST /token para trocar o código

Por isso você vê, logo depois, o frame #3287 com:

Frame 3287 – POST /token**❑ USANDO POSTMAN**

Aqui é onde o Postman faz a troca do código por um access token, usando o PKCE:

No corpo da requisição são enviados:

- grant_type=authorization_code
- code (o que veio no redirecionamento)
- client_id
- redirect_uri
- code_verifier (chave PKCE real, que valida o code_challenge usado lá atrás)

Esse é o ponto mais crítico da segurança do fluxo Authorization Code com PKCE.

Aqui é importante destacar o papel dos vários tipos de códigos que temos neste fluxo de autenticação:

Elemento	Tipo / Origem	Quem cria?	Vai para onde?	Para que serve?
code	Authorization Code	Keycloak	Retornado via redirect_uri (302)	Usado para trocar por tokens reais no POST /token
code_verifier	PKCE (RFC 7636)	Client (ex: SPA)	Gerado no frontend	Enviado no POST /token para comprovar que quem trocou o code é o mesmo cliente
code_challenge	PKCE	Client	Vai na URL de autorização inicial (/auth)	É a versão hash do code_verifier, que Keycloak vai usar para validar o verifier

O fluxo seria assim:

1. POSTMAN gera:


```
code_verifier = string aleatória
code_challenge = SHA256(code_verifier)
```
2. Redireciona o usuário → requisição Keycloak:


```
GET /realms/myrealm/openid-connect/auth?...&code_challenge=xyz&code_challenge_method=S256
```
3. Usuário se autentica → Keycloak responde HTTP 302:

302 Location: <http://localhost:????/callback?code=abc123&state=xyz> (Postman intercepta e captura o code, por isso a porta NESTE CASO é irrelevante)
4. POSTMAN captura `code` e envia requisição POST→ Keycloak, note que agora que POSTMAN vai enviar code_verifier, Keycloak vai comparar o hash de code_verifier com o hash recebido de code_challenge, se for igual, para garantir que ninguém capture o code e use de forma maliciosa:


```
POST /token
{
  grant_type=authorization_code,
  code=abc123,
  code_verifier=original_string,
  ...
}
```
5. Keycloak valida:


```
SHA256(code_verifier) == code_challenge?
```

Se sim → emite os tokens

Para compreender melhor o fluxo, imagine um cofre com senha:

- o O POSTMAN gera a senha (`code_verifier`). Na realidade entramos manualmente com o `code_verifier`.
- o Coloca-se o hash da senha (`code_challenge`) no cofre (Keycloak)
- o Depois, quando quiser abrir (trocar `code` por `token`), POSTMAN apresenta a senha real (`code_verifier`)
- o Se o hash bate com o que foi guardado, Keycloak entrega os tokens

❑ FLUXO REAL OAuth2 COM PKCE (FRONTEND/BACKEND)

Componente	Função	Exemplo
Keycloak	Authorization Server (emite tokens)	http://localhost:8081
Frontend (SPA)	Client OAuth2 + UI (ex: React, Angular)	http://localhost:3000
Backend (API)	Resource Server protegido (ex: Spring)	http://localhost:8080

1. Usuário acessa o frontend

<http://localhost:3000>

2. Frontend redireciona para Keycloak:

```
GET http://localhost:8081/realms/myrealm/protocol/openid-connect/auth?response_type=code
&client_id=myclient
&redirect_uri=http://localhost:3000/callback
&code_challenge=...
```

3. Usuário faz login na tela do Keycloak**4. Keycloak redireciona de volta:**

302 Found → Location: <http://localhost:3000/callback?code=abc123&state=xyz>

⚠ IMPORTANTE:

Keycloak não faz uma requisição POST para essa URI. Ele faz um redirect HTTP (302) via navegador. O navegador segue o redirecionamento automaticamente, e faz um GET para `redirect_uri`.

GET <http://localhost:3000/callback?code=abc123&state=xyz>

Este é o comportamento padrão dos navegadores quando encontram um código de status HTTP 302 - eles fazem automaticamente uma requisição GET para o endereço especificado no cabeçalho "Location". Esse mecanismo é fundamental para o fluxo Authorization Code, pois é assim que o código de autorização chega até o frontend (aplicação cliente), que depois pode trocá-lo por tokens usando o endpoint /token.

5. Frontend captura o code e faz o POST /token direto do browser (PKCE)

```
POST http://localhost:8081/realms/myrealm/protocol/openid-connect/token
Content-Type: application/x-www-form-urlencoded
grant_type=authorization_code
code=abc123
```

```
redirect_uri=http://localhost:3000/callback
client_id=myclient
code_verifier=...
```

6. Keycloak responde com os tokens:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs... ",
  "refresh_token": "...",
  "id_token": "...",
  "expires_in": 300,
  "token_type": "Bearer"
}
```

7. Frontend agora acessa a API protegida:

GET <http://localhost:8080/clients>
Authorization: Bearer eyJhbGciOi...

- o O backend (Resource Server) **valida o access_token**
- o Se válido, **retorna os dados protegidos** (ex: lista de clientes)

a. Validação do token

Se o backend usa **Spring Security + OAuth2 Resource Server**, ele validará automaticamente o token:

- Usando a issuer-uri (<http://localhost:8081/realms/myrealm>)
- Validando assinatura, expiração, escopo, etc.

Resumindo:

Etapa	Responsável	Resultado
Autenticação do usuário	Frontend + Keycloak	Código de autorização (code)
Troca de code por token	Frontend	Recebe access_token, id_token, refresh_token
Acesso ao backend (/clients)	Frontend → Backend	Envia Authorization: Bearer <token>
Validação do token	Backend (Resource)	Verifica via chave pública do Keycloak

Frame 3289 – HTTP 200 OK + JSON

O Keycloak responde com:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9....",
  "expires_in": 300,
  "refresh_expires_in": 1800,
  "refresh_token": "...",
  "token_type": "Bearer",
  "not-before-policy": 0,
  "session_state": "...",
  "scope": "openid"
}
```

Obteve-se com sucesso o **access_token** e o **refresh_token**. Isso completa o ciclo do Authorization Code com PKCE.

Resumo Visual do Fluxo (frames 3253 → 3289)

